

Assignment 2 - EECS 348 (Software Engineering) by Lucas Frias

Welcome back, I'm excited for this assignment. Last time, we worked on the basic premise of the AI analysis of the code, and I chose two lesser models. When discussing this with my peers, they told me that Anthropic and OpenAI's latest models, in comparison to the attempts that the local Gwen 4B model and the novel Apple Intelligence, would be like apples to oranges. Both of these companies seem to show a great amount of investment and control in the modern tech landscape, and each are trying to compete to remain the dominant AI company of the future. I decided to do what I had avoided before, and use the two newer AI models.

My access model was to visit the websites hosted by these AI companies, create an account, and enter in the prompt. I decided to use Sonnet 5 by Anthropic (on their website [Claude.ai](https://claude.ai)) and OpenAI's GPT 5.6 Luna (on the website chatgpt.com) and pitted them against one another in this project.

I will be referring to Anthropic's model as Sonnet and OpenAI's as Luna throughout this document.

The prompt is modified from the original assignment text, so here it is below:

Make this program in C.

- The program will prioritize emails for a busy company CEO.
- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create functions from scratch. Do not include pre-existing heap modules.

Here's the file format:

EMAIL <sender category>, <subject line>, <date> - The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.

EMAIL is followed by space. The rest of the fields are delimited. Assume <sender category> is one of the five strings listed above. Assume <subject line> is a string which may contain spaces, but not commas. Assume <date> is in the format: MM-DD-YYYY

NEXT - Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:

Date:

READ - CEO has read with the email and has dealt with it

COUNT - display current count of untracked files

Here's an example file:

EMAIL Peer,Can you help me on this?,12-01-2024

EMAIL OtherPerson,Try our product,12-19-2024

EMAIL Boss,Important,12-20-2024

EMAIL Subordinate,How do I handle this?,12-25-2024

EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024

EMAIL Boss,Never Mind,01-03-2025

COUNT

NEXT

READ

NEXT

READ

COUNT

Here is the output that the program should give

There are 6 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important

Date: 12-20-2024

There are 4 emails to read.

Good luck!

The only modification that has been done are slight markdown changes to format into this document. You can see the original in PROMPT.md.

Below are the analysis pertaining to each category below. I hope this is informative and correct.

OpenAI'S Luna Ouput:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_EMAILS 1000
#define MAX_SUBJECT 500
#define MAX_CATEGORY 30

/*
    Higher number = higher priority.
    Boss is highest priority, OtherPerson is lowest.
*/
int getCategoryPriority(const char *category)
{
    if (strcmp(category, "Boss") == 0)
        return 5;
    else if (strcmp(category, "Subordinate") == 0)
        return 4;
    else if (strcmp(category, "Peer") == 0)
        return 3;
    else if (strcmp(category, "ImportantPerson") == 0)
        return 2;
    else if (strcmp(category, "OtherPerson") == 0)
        return 1;

    return 0;
}

/* Convert MM-DD-YYYY into YYYYMMDD for easy date comparison. */
int convertDate(const char *date)
{
    int month, day, year;

    sscanf(date, "%d-%d-%d", &month, &day, &year);

    return year * 10000 + month * 100 + day;
}

```

```

/* An email stored inside the heap. */
typedef struct
{
    char sender[MAX_CATEGORY];
    char subject[MAX_SUBJECT];
    char date[20];

    int categoryPriority;
    int dateValue;
} Email;

/* MaxHeap implemented using an array/list. */
typedef struct
{
    Email emails[MAX_EMAILS];
    int size;
} MaxHeap;

/*
Returns 1 if email1 should appear before email2.

Priority rules:
1. Higher sender category priority comes first.
2. If categories are equal, newer date comes first.
*/
int higherPriority(Email email1, Email email2)
{
    if (email1.categoryPriority > email2.categoryPriority)
        return 1;

    if (email1.categoryPriority < email2.categoryPriority)
        return 0;

    /* Same sender category: newest email comes first. */
    if (email1.dateValue > email2.dateValue)

```

```

        return 1;

    return 0;
}

/* Swap two emails in the heap. */
void swapEmails(Email *email1, Email *email2)
{
    Email temp = *email1;
    *email1 = *email2;
    *email2 = temp;
}

/*
    Move an email upward in the heap until the MaxHeap
    property is restored.
*/
void heapifyUp(MaxHeap *heap, int index)
{
    int parent;

    while (index > 0)
    {
        parent = (index - 1) / 2;

        if (higherPriority(heap->emails[index], heap->emails[parent]))
        {
            swapEmails(&heap->emails[index],
                       &heap->emails[parent]);

            index = parent;
        }
        else
        {
            break;
        }
    }
}

```

```

    }
}

/*
    Move an email downward in the heap until the MaxHeap
    property is restored.
*/
void heapifyDown(MaxHeap *heap, int index)
{
    int left;
    int right;
    int largest;

    while (1)
    {
        left = 2 * index + 1;
        right = 2 * index + 2;
        largest = index;

        /* Check left child. */
        if (left < heap->size &&
            higherPriority(heap->emails[left],
                          heap->emails[largest]))
        {
            largest = left;
        }

        /* Check right child. */
        if (right < heap->size &&
            higherPriority(heap->emails[right],
                          heap->emails[largest]))
        {
            largest = right;
        }
    }
}

```

```

        /* No child has higher priority. */
        if (largest == index)
            break;

        swapEmails(&heap->emails[index],
                  &heap->emails[largest]);

        index = largest;
    }
}

/* Initialize an empty heap. */
void initializeHeap(MaxHeap *heap)
{
    heap->size = 0;
}

/* Add an email to the MaxHeap. */
void insertEmail(MaxHeap *heap, Email email)
{
    if (heap->size >= MAX_EMAILS)
    {
        printf("Error: Maximum number of emails reached.\n");
        return;
    }

    /* Put the new email at the end of the list. */
    heap->emails[heap->size] = email;

    /* Move it upward into the correct position. */
    heapifyUp(heap, heap->size);

    heap->size++;
}

```

```

/*
    Remove and return the highest-priority email.
    This is the MaxHeap delete operation.
*/
Email removeMax(MaxHeap *heap)
{
    Email result;

    /* Safety value if heap is empty. */
    result.sender[0] = '\0';
    result.subject[0] = '\0';
    result.date[0] = '\0';

    if (heap->size == 0)
        return result;

    /* The root is the highest-priority email. */
    result = heap->emails[0];

    /* Move the last email to the root. */
    heap->size--;

    if (heap->size > 0)
    {
        heap->emails[0] = heap->emails[heap->size];

        /* Restore MaxHeap property. */
        heapifyDown(heap, 0);
    }

    return result;
}

/*
    Look at the highest-priority email without removing it.

```

```

*/
Email peekMax(MaxHeap *heap)
{
    Email result;

    result.sender[0] = '\0';
    result.subject[0] = '\0';
    result.date[0] = '\0';

    if (heap->size == 0)
        return result;

    return heap->emails[0];
}

/*
    Parse an EMAIL command.

    Example:
    EMAIL Peer,Can you help me on this?,12-01-2024
*/
Email parseEmail(char *line)
{
    Email email;

    char *data;
    char *sender;
    char *subject;
    char *date;

    /* Skip "EMAIL " */
    data = line + 6;

    /* Get sender category. */
    sender = strtok(data, ",");

```

```
/* Get subject. */
subject = strtok(NULL, ",");

/* Get date. */
date = strtok(NULL, "\\n");

if (sender == NULL)
    sender = "";

if (subject == NULL)
    subject = "";

if (date == NULL)
    date = "";

/* Remove possible spaces around fields. */
while (*sender == ' ')
    sender++;

while (*subject == ' ')
    subject++;

while (*date == ' ')
    date++;

/* Copy information into the Email structure. */
strncpy(email.sender, sender, MAX_CATEGORY - 1);
email.sender[MAX_CATEGORY - 1] = '\\0';

strncpy(email.subject, subject, MAX_SUBJECT - 1);
email.subject[MAX_SUBJECT - 1] = '\\0';

strncpy(email.date, date, sizeof(email.date) - 1);
email.date[sizeof(email.date) - 1] = '\\0';

/* Remove trailing carriage return if the file uses Windows line
endings. */
```

```

    email.date[strcspn(email.date, "\r\n")] = '\0';

    email.categoryPriority = getCategoryPriority(email.sender);
    email.dateValue = convertDate(email.date);

    return email;
}

/* Display the next email without removing it. */
void nextEmail(MaxHeap *heap)
{
    Email email;

    if (heap->size == 0)
    {
        printf("There are no emails to read.\n");
        return;
    }

    email = peekMax(heap);

    printf("Next email:\n");
    printf("Sender: %s\n", email.sender);
    printf("Subject: %s\n", email.subject);
    printf("Date: %s\n", email.date);
}

/* READ removes the highest-priority email. */
void readEmail(MaxHeap *heap)
{
    if (heap->size == 0)
    {
        printf("There are no emails to read.\n");
        return;
    }
}

```

```

        removeMax(heap);
    }

/* Display the current number of emails in the heap. */
void countEmails(MaxHeap *heap)
{
    printf("There are %d emails to read.\n", heap->size);
}

int main(int argc, char *argv[])
{
    FILE *file;
    char line[600];

    MaxHeap heap;

    initializeHeap(&heap);

    /*
        The file name can be supplied when running the program:

        ./email_priority emails.txt
    */
    if (argc < 2)
    {
        printf("Usage: %s <input file>\n", argv[0]);
        return 1;
    }

    file = fopen(argv[1], "r");

    if (file == NULL)
    {
        printf("Error: Could not open file %s\n", argv[1]);
    }
}

```

```
        return 1;
    }

    /* Read each command from the input file. */
    while (fgets(line, sizeof(line), file) != NULL)
    {
        /* Remove newline characters. */
        line[strcspn(line, "\r\n")] = '\0';

        /* Ignore blank lines. */
        if (strlen(line) == 0)
            continue;

        /* EMAIL command */
        if (strncmp(line, "EMAIL ", 6) == 0)
        {
            Email email = parseEmail(line);
            insertEmail(&heap, email);
        }

        /* NEXT command */
        else if (strcmp(line, "NEXT") == 0)
        {
            nextEmail(&heap);
        }

        /* READ command */
        else if (strcmp(line, "READ") == 0)
        {
            readEmail(&heap);
        }

        /* COUNT command */
        else if (strcmp(line, "COUNT") == 0)
        {
            countEmails(&heap);
        }
    }
}
```

```
}  
  
fclose(file);  
  
return 0;  
}
```

Anthropic's Sonnet (Medium) Output:

```

/* =====
 * email_priority.c
 *
 * Prioritizes a busy CEO's inbox using a MaxHeap (array/list based
 * priority queue) implemented entirely from scratch -- no library
 * heap functions are used.
 *
 * Commands read from a script file (given as argv[1]) or stdin:
 *
 * EMAIL <sender category>,<subject line>,<date MM-DD-YYYY>
 * NEXT
 * READ
 * COUNT
 *
 * Sender category priority (highest to lowest):
 * Boss > Subordinate > Peer > ImportantPerson > OtherPerson
 *
 * Within the same category, the NEWEST email (largest date) is
 * read first.
 * ===== */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

/* -----
 * Constants
 * ----- */
#define MAX_SUBJECT_LEN 256
#define MAX_CATEGORY_LEN 32
#define INITIAL_CAPACITY 16

/* -----
 * Email struct
 * ----- */
typedef struct {

```

```

    char category[MAX_CATEGORY_LEN]; /* Boss, Subordinate, Peer, ...
*/
    char subject[MAX_SUBJECT_LEN]; /* subject line
*/
    char date[11]; /* MM-DD-YYYY (10 chars + NUL)
*/
    int categoryRank; /* numeric priority of category
*/
    long dateValue; /* YYYYMMDD, for easy comparison
*/
} Email;

/* -----
 * MaxHeap struct (list/array based implementation)
 * ----- */
typedef struct {
    Email *data; /* dynamically allocated array of emails */
    int size; /* number of elements currently in heap */
    int capacity; /* allocated capacity of the array */
} MaxHeap;

/* =====
 * Category priority lookup
 *
 * Higher number == higher priority (read sooner).
 * ===== */
int categoryRank(const char *category) {
    if (strcmp(category, "Boss") == 0) return 5;
    if (strcmp(category, "Subordinate") == 0) return 4;
    if (strcmp(category, "Peer") == 0) return 3;
    if (strcmp(category, "ImportantPerson") == 0) return 2;
    if (strcmp(category, "OtherPerson") == 0) return 1;
    return 0; /* unknown category - lowest priority */
}

/* =====
 * Convert MM-DD-YYYY into a comparable integer YYYYMMDD.

```

```

* Larger value == newer date.
* ===== */
long dateToValue(const char *date) {
    int month = 0, day = 0, year = 0;
    if (sscanf(date, "%d-%d-%d", &month, &day, &year) != 3) {
        return 0; /* malformed date, treat as very old */
    }
    return ((long)year * 10000L) + (month * 100L) + day;
}

/* =====
* Compare two emails for max-heap ordering.
* Returns 1 if "a" has strictly higher priority than "b", else 0.
*
* Priority rules:
* 1) Higher categoryRank wins.
* 2) If categories tie, newer date (larger dateValue) wins.
* ===== */
int emailHasHigherPriority(const Email *a, const Email *b) {
    if (a->categoryRank != b->categoryRank) {
        return a->categoryRank > b->categoryRank;
    }
    return a->dateValue > b->dateValue;
}

/* =====
* MaxHeap creation / destruction
* ===== */
MaxHeap *heapCreate(int initialCapacity) {
    MaxHeap *heap = (MaxHeap *)malloc(sizeof(MaxHeap));
    if (!heap) {
        fprintf(stderr, "Fatal: out of memory creating heap.\n");
        exit(EXIT_FAILURE);
    }
    heap->data = (Email *)malloc(sizeof(Email) * initialCapacity);
    if (!heap->data) {
        fprintf(stderr, "Fatal: out of memory allocating heap

```

```

storage.\n");
    exit(EXIT_FAILURE);
}
heap->size = 0;
heap->capacity = initialCapacity;
return heap;
}

void heapDestroy(MaxHeap *heap) {
    if (!heap) return;
    free(heap->data);
    free(heap);
}

/* Double the backing array when it fills up. */
void heapEnsureCapacity(MaxHeap *heap) {
    if (heap->size < heap->capacity) return;
    int newCapacity = heap->capacity * 2;
    Email *newData = (Email *)realloc(heap->data, sizeof(Email) *
newCapacity);
    if (!newData) {
        fprintf(stderr, "Fatal: out of memory growing heap.\n");
        exit(EXIT_FAILURE);
    }
    heap->data = newData;
    heap->capacity = newCapacity;
}

/* -----
 * Index helpers for the array-based (list-based) heap layout.
 * For node at index i (0-based):
 *   parent  = (i - 1) / 2
 *   left    = 2*i + 1
 *   right   = 2*i + 2
 * ----- */
static int parentIndex(int i) { return (i - 1) / 2; }
static int leftChildIndex(int i) { return 2 * i + 1; }

```

```

static int rightChildIndex(int i) { return 2 * i + 2; }

static void swapEmails(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

/* -----
 * Sift-up (bubble-up): restores heap property after insertion
 * at the end of the array.
 * ----- */
void heapSiftUp(MaxHeap *heap, int index) {
    while (index > 0) {
        int parent = parentIndex(index);
        if (emailHasHigherPriority(&heap->data[index], &heap-
>data[parent])) {
            swapEmails(&heap->data[index], &heap->data[parent]);
            index = parent;
        } else {
            break;
        }
    }
}

/* -----
 * Sift-down (bubble-down): restores heap property after the
 * root is replaced (e.g. during extraction).
 * ----- */
void heapSiftDown(MaxHeap *heap, int index) {
    while (1) {
        int left = leftChildIndex(index);
        int right = rightChildIndex(index);
        int largest = index;

        if (left < heap->size &&
            emailHasHigherPriority(&heap->data[left], &heap-

```

```

>data[largest])) {
    largest = left;
}
if (right < heap->size &&
    emailHasHigherPriority(&heap->data[right], &heap-
>data[largest])) {
    largest = right;
}
if (largest == index) break;

swapEmails(&heap->data[index], &heap->data[largest]);
index = largest;
}
}

/* -----
 * Insert a new email into the heap.
 * ----- */
void heapInsert(MaxHeap *heap, Email email) {
    heapEnsureCapacity(heap);
    heap->data[heap->size] = email;
    heapSiftUp(heap, heap->size);
    heap->size++;
}

/* -----
 * Peek at the highest-priority email without removing it.
 * Returns 1 on success, 0 if heap is empty.
 * ----- */
int heapPeek(MaxHeap *heap, Email *outEmail) {
    if (heap->size == 0) return 0;
    *outEmail = heap->data[0];
    return 1;
}

/* -----
 * Remove and return the highest-priority email.

```

```

* Returns 1 on success, 0 if heap is empty.
* ----- */
int heapExtractMax(MaxHeap *heap, Email *outEmail) {
    if (heap->size == 0) return 0;

    *outEmail = heap->data[0];
    heap->size--;

    if (heap->size > 0) {
        heap->data[0] = heap->data[heap->size];
        heapSiftDown(heap, 0);
    }
    return 1;
}

int heapSize(MaxHeap *heap) {
    return heap->size;
}

/* =====
* String helpers
* ===== */

/* Strip trailing newline / carriage return characters. */
void trimNewline(char *s) {
    size_t len = strlen(s);
    while (len > 0 && (s[len - 1] == '\n' || s[len - 1] == '\r')) {
        s[len - 1] = '\0';
        len--;
    }
}

/* Remove leading/trailing whitespace in place. */
void trimWhitespace(char *s) {
    /* Trim trailing */
    int len = (int)strlen(s);
    while (len > 0 && isspace((unsigned char)s[len - 1])) {

```

```

        s[--len] = '\0';
    }
    /* Trim leading */
    int start = 0;
    while (s[start] != '\0' && isspace((unsigned char)s[start])) {
        start++;
    }
    if (start > 0) {
        memmove(s, s + start, strlen(s + start) + 1);
    }
}

/* =====
 * Parse an "EMAIL <category>,<subject>,<date>" line (the text
 * AFTER the leading "EMAIL " token has already been stripped
 * off by the caller) into an Email struct.
 *
 * Returns 1 on success, 0 on a malformed line.
 * ===== */
int parseEmailFields(char *fields, Email *outEmail) {
    /* fields looks like: "Boss,Never Mind,01-03-2025" */
    char *firstComma = strchr(fields, ',');
    if (!firstComma) return 0;
    *firstComma = '\0';
    char *category = fields;

    char *rest = firstComma + 1;
    char *secondComma = strrchr(rest, ','); /* subject can't have
commas,
so the LAST comma in the
remainder separates the
subject from the date */

    if (!secondComma) return 0;
    *secondComma = '\0';
    char *subject = rest;
    char *date = secondComma + 1;

```

```

    trimWhitespace(category);
    trimWhitespace(subject);
    trimWhitespace(date);

    if (strlen(category) == 0 || strlen(date) == 0) return 0;

    strncpy(outEmail->category, category, MAX_CATEGORY_LEN - 1);
    outEmail->category[MAX_CATEGORY_LEN - 1] = '\0';

    strncpy(outEmail->subject, subject, MAX_SUBJECT_LEN - 1);
    outEmail->subject[MAX_SUBJECT_LEN - 1] = '\0';

    strncpy(outEmail->date, date, sizeof(outEmail->date) - 1);
    outEmail->date[sizeof(outEmail->date) - 1] = '\0';

    outEmail->categoryRank = categoryRank(outEmail->category);
    outEmail->dateValue = dateToValue(outEmail->date);

    return 1;
}

/* =====
 * Command handlers
 * ===== */
void handleEmailCommand(MaxHeap *heap, char *line) {
    /* line begins with "EMAIL ", the rest is the field data */
    char *fields = line + strlen("EMAIL");
    while (*fields == ' ') fields++; /* skip the single space (and any
extra) */

    Email email;
    if (!parseEmailFields(fields, &email)) {
        fprintf(stderr, "Warning: could not parse EMAIL line: %s\n",
line);
        return;
    }
    heapInsert(heap, email);
}

```

```

}

void handleNextCommand(MaxHeap *heap) {
    Email top;
    if (!heapPeek(heap, &top)) {
        printf("Next email:\n");
        printf("There are no emails to read.\n\n");
        return;
    }
    printf("Next email:\n");
    printf("Sender: %s\n", top.category);
    printf("Subject: %s\n", top.subject);
    printf("Date: %s\n\n", top.date);
}

void handleReadCommand(MaxHeap *heap) {
    Email dequeued;
    if (!heapExtractMax(heap, &dequeued)) {
        printf("There is nothing to mark as read.\n\n");
    }
    /* Per spec, READ simply removes the current top email
       (the CEO dealt with it). No output is required. */
}

void handleCountCommand(MaxHeap *heap) {
    printf("There are %d emails to read.\n\n", heapSize(heap));
}

/* =====
 * Main driver: reads commands line by line from the given file
 * (or stdin if no file argument is provided) and dispatches
 * them to the appropriate handler.
 * ===== */
int main(int argc, char *argv[]) {
    FILE *input = stdin;

    if (argc > 1) {

```

```

    input = fopen(argv[1], "r");
    if (!input) {
        fprintf(stderr, "Error: could not open file '%s'\n",
argv[1]);
        return EXIT_FAILURE;
    }
}

MaxHeap *heap = heapCreate(INITIAL_CAPACITY);

char line[1024];
while (fgets(line, sizeof(line), input)) {
    trimNewline(line);

    /* Skip blank lines */
    char *check = line;
    while (*check == ' ' || *check == '\t') check++;
    if (*check == '\0') continue;

    if (strncmp(line, "EMAIL", 5) == 0) {
        handleEmailCommand(heap, line);
    } else if (strncmp(line, "NEXT", 4) == 0) {
        handleNextCommand(heap);
    } else if (strncmp(line, "READ", 4) == 0) {
        handleReadCommand(heap);
    } else if (strncmp(line, "COUNT", 5) == 0) {
        handleCountCommand(heap);
    } else {
        fprintf(stderr, "Warning: unrecognized command: %s\n",
line);
    }
}

if (input != stdin) {
    fclose(input);
}

heapDestroy(heap);

```

```
    return EXIT_SUCCESS;
}
```

Correctness

Issues were immediately encountered here. OpenAI's Luna model was unable to produce functioning code, and as per the assignment instructions I cannot ask for further clarification. Nor am I going to cherry-pick the best result given by OpenAI to try and get better results.

The Spacing Problem

The mistake was a slight one. In the main program execution, after accepting a file as input, Luna's code reads the file correctly. It even is able to add elements to the MaxHeap:

```
...
/* EMAIL command */
if (strncmp(line, "EMAIL ", 6) == 0) //correct spacing
{
    Email email = parseEmail(line);
    insertEmail(&heap, email);
}
...
```

There's two details important here. First is the spacing. Since we probably want to have spacing between our commands in the file formatting, it's important to include the spacing. However, Luna fails to do this and switches between two similar functions **strncmp** versus **strcmp**.

```

else if (strcmp(line, "NEXT") == 0)
{
    nextEmail(&heap);
}

/* READ command */
else if (strcmp(line, "READ") == 0)
{
    readEmail(&heap);
}

```

According to Mr Bharati of StackOverflow fame

(<https://stackoverflow.com/questions/30190460/advantages-of-strncmp-over-strcmp#30190652>) he has this to say about the difference:

The problem with strcmp is that sometimes, if by mistake, arguments that are passed are not valid C-strings (meaning that p1 or p2 is not terminated with a null character i.e. not NULL-terminated String), then, strcmp continues comparing until it reaches non-accessible memory and crashes or sometimes results to an unexpected behaviour.

Is that what happens here? Yes, unfortunately. strcmp does not keep going until it sees the null terminator character, but it's very unsafe because the input file doesn't guarantee that the newline character is this. This causes every single strcmp to fail. Sonnet avoids this pitfall, but Luna fails with just a single space occurring after the line. strncmp would be much safer here because for each newline the promise is that there is at least some command such as COUNT, READ, NEXT. Just adding strncmp with the correct sizing fixes this bug simply, but Luna was unable to anticipate this, partly because of the file format but mostly because of using unsafe functions and trusting they work and ignoring new space.

The file format had padding on the actual prompt given (there is a space 31 0x0a character) and Luna just ignored it and believed that there would never be padding. Luna knows how to use strncmp as they do with the EMAIL portion, but fails to actually use it correctly for the other edge cases.

I took the liberty of fixing this bug and then testing Sonnet's and Luna's output with the test file. Both Sonnet and Luna corrected outputted fine, but Luna unpatched outputted nothing because it did not recognize any commands when there was one space at the end of the command. I will still be analyzing Luna's output in terms of time complexity, space

complexity, and maintainability, because despite this error the implementation works for the test file. However, note that this analysis works only for when files are specially formatted for Luna's preferred formatting. This makes it so that Luna has less correct output than Sonnet.

Quick note: I believe Sonnet predicted this as it was able to test its code within a Unix-like shell given to it by Anthropic, anticipate the space, and program in edge cases.

Heap VS Stack Prompting Analysis

The prompt I gave, derived from the assignment prompt, specified

```
You will use a MaxHeap as a means of implementing a priority queue
```

and then later stated

```
You must implement a MaxHeap using a list-based implementation.
```

the key word being "list".

One very commonly discussed feature with C and other statically strictly typed languages is the fact that array sizes are known at run time, or when the array is created. It is not recommended and is UB to add extra elements to an array after it is created.

```
char a[32] = {0}; //make a char array to do cstring stuff
a[1] = 'a'; //fine and dandy
a[33] = 12; //UB!! maybe this works, maybe we segfault, maybe we
overwrite some other stack variable.
```

If we want to still do this in the stack, there's a couple of ways.

Firstly, we can just use a really big stack frame

```
Email email_max_heap[MAX_EMAIL_SIZE] = {0};
```

We do use a lot of stack that we don't probably need, but modern operating systems are smart with these things and actually don't really give you the memory unless you really need it. However, this might not be the best practice. There are other ways, because we don't have to set the max_heap_size at compile time.

If we are trying to, for example, track these emails from the file, we can't know the possible amount of emails before we read them. We could do something like this:

```
//Note: this is psuedo code, and I haven't actually tested it for
unsafeness
//just to illustrate my point. Also thank you to AProgrammer from
Stack0verflow
//https://stackoverflow.com/questions/9206091/going-through-a-text-
file-line-by-line-in-c
FILE * fptr = open("EMAILS.TXT", "rb");//grab our fptr
char cmd_buf[512] = {0};
unsigned int email_count = 0;
while (*fptr!=NULL){
    fgets(cmd_buf, sizeof(cmd_buf), fptr);
    if (! (strncmp(cmd_buf, "EMAIL") )){ //if strncmp doesn't return
any differences
        email_count++; //increment the email count
    }
}
Email * email_max_heap_arr[email_count] = {0}; //make a new maxheap
with the email array account. we can still malloc email structs, but use
an array
```

This snippet of code I programmed up there would be smarter than just setting some max size, but also would take an extra pass before evaluation, possibly increasing the time complexity, and also could still have unused memory once an email is read and "de-heaped" from the array. This would contain extra unused memory.

If we instead want to have a completely dynamic implementation, we have to use the heap and allocate variables. We could do this two ways: with a linked list or an array.

However, when doing max heaps, it would be better to use an array based implementation instead of a linked list because each malloc for a new node in a linked list will give it a completely unrelated virtual memory address. An array is the best way, but it because difficult. Look at this function below:

```
void heapEnsureCapacity(MaxHeap *heap) {
    if (heap->size < heap->capacity) return;
    int newCapacity = heap->capacity * 2;
    Email *newData = (Email *)realloc(heap->data, sizeof(Email) *
newCapacity);
    if (!newData) {
        fprintf(stderr, "Fatal: out of memory growing heap.\n");
        exit(EXIT_FAILURE);
    }
    heap->data = newData;
    heap->capacity = newCapacity;
}
```

This is a function used by Sonnet's implementation to ensure before an addition that the new size of the array is large enough to actually add the element. If not, it will double in size. However, we want to guarantee contiguous (touching) memory, so we use realloc for the heap data and then pass the new capacity to the correct position.

This is a very fancy solution, but it's kind of like bringing silverware to a McDonald's. A stack based implementation is possible without mallocing like this, but it is a pretty implementation too. The devil is in the details of whether or not we need to worry about massive amounts of emails. Let's do the math:

In 2009 (<https://stackoverflow.com/questions/1825964/c-c-maximum-stack-size-of-program-on-mainstream-oses#1825996>) the average stack frame size is roughly ~ 1 MB.

Our email struct is:

```
typedef struct {
    char category[MAX_CATEGORY_LEN]; /* Boss, Subordinate, Peer, ...
*/
    char subject[MAX_SUBJECT_LEN]; /* subject line
*/
    char date[11]; /* MM-DD-YYYY (10 chars + NUL)
*/
    int categoryRank; /* numeric priority of category
*/
    long dateValue; /* YYYYMMDD, for easy comparison
*/
} Email;
```

where:

```
#define MAX_SUBJECT_LEN    256
#define MAX_CATEGORY_LEN  32
#define INITIAL_CAPACITY  16
```

(Small note, it is strange that the date is a string AND a number, at least for readability. We'll talk about this soon)

Anyways, we can calculate roughly how much the struct's size ought to be.

$\text{char} * 256 + \text{char} * 32 + \text{char} * 11 + \text{int} + \text{long}$

$256 + 32 + 11 + 4 + 8$ //note here, i'm assuming modern implementations where int is 4

$256 + 32 + 11 + 4 + 8 = \mathbf{311}$ bytes per email

Assume we have a pittance of 1 megabyte of stack frame. Let's calculate the worst case scenario (assume that the rest of our program uses 1 512 bytes for misc.)

$(1,000,000 - 512) / 311 \approx 3,120$ emails possible to be processed on one stack frame.

Being a KU student for a year and a half has given me 1,526 emails in my mailbox. I promise you, KU students get a ridiculous amount of emails. Maybe not as much as a CEO, but I think it's comparable

So, theoretically, if our boss was running his same email server on Windows Vista on a Pentium, he would have to clean his inbox once a year. Hopefully he's not. But since this is corporate software, expecting people to upgrade their systems and follow maintainance is a miracle. A heap based implementation probably is the way to go, and even though this is a lot of analysis, this is a fundamental decision choice. Heap isn't that much worse or better than stack, but it is interesting, because I believe that most human C programmers would probably do a heap implementation.

Can't say too much else about the MaxHeap implementation. AI has been programmed on thousands of maxheap implementations, and it chose a decent one for C.

Time Complexity

There's a lot to analyze on this, but here's the flow of each program, generally

General Program Flow:

- Read the file from argc
- Go through every character, and handle the command:
 - If the command == EMAIL, heapInsert
 - Then heapSiftUp
 - while loop to set the index at the right place
 - If the command == READ, then use HeapExtractMax
 - if the size is zero then return
 - else, heapSiftDown
 - while loop for every element that sorts everything

This is pretty much just a MaxHeap implementation. Since this is a type of complete Binary Tree, this has a time complexity of $O(n * \log(n))$ complexity, and a space complexity of $O(n)$ for every element. We'll touch on space later.

Real Runtime Comparisons:

Checking the actual runtime with the **time** command:

Sonnet's Real Time Complexity

```
lucasfrias@Frias-iMac Fall2026_EECS348_Assignment_2 % time ./claude
DEMO.txt
There are 3 emails to read.

There are 4 emails to read.

Next email:
Sender: Boss
Subject: Never Mind
Date: 01-03-2025

Next email:
Sender: Boss
Subject: Important, Mind
Date: 01-03-2025

There are 2 emails to read.

./claude DEMO.txt  0.05s user 0.02s system 95% cpu 0.072 total
```

Luna's Real Time Complexity

For the d_openai.c (REMEMBER, this is the modified one that accepts the normal file input with whitespacing, but besides that functionally the same):

```
lucasfrias@Frias-iMac Fall2026_EECS348_Assignment_2 % time ./d_openai
DEMO.txt
There are 3 emails to read.
There are 4 emails to read.
Next email:
Sender: Boss
Subject: Never Mind
Date: 01-03-2025
Next email:
Sender: Boss
Subject: Never Mind
Date: 01-03-2025
There are 3 emails to read.
./d_openai DEMO.txt 0.05s user 0.02s system 96% cpu 0.069 total
```

As you can see, the time differs very little between one or the other, by the millisecond. I believe that 's is "faster" because it doesn't do character parsing, and instead is just assuming there's a certain amount of whitespace.

Extra Malloc Time

There is also not too much to say on the MaxHeap complexity, expect that because it's heap allocated, it is ever so slightly slower because of the heap. This is because unlike dynamically sized linked lists or binary search trees, the maxheap can generally be done in an array (remember above in Correctness the heap versus stack discussion as it relates to MaxHeap)

From Mr Delroy of StackOverflow:

(<https://stackoverflow.com/questions/24057331/is-accessing-data-in-the-heap-faster-than-from-the-stack>)

So, when using the stack programmers tend to do what they can with arrays, which are contiguous in memory, even if it means a little brute-force searching. The cache-efficiency may well make this better overall than heap based data containers where the elements are spread across more cache lines. Of course, stack usage doesn't scale to large numbers of elements, and - without at least a backup option of using heap - creates programs that stop working if given more data to process than expected.

P.S. I highly encourage you to read this post, it is insightful.

Exactly the point to illustrate, the heap allocation will just be a bit slower. We can call this delay in allocation M . Sonnet's default macro for the allocation size is 16, so we can say that if it passes through this, we need to reallocate the the memory, which doubles it's capacity. Therefore, the total delay T for a maximum occurance of reallocation N is:

$$T = M + M*N$$

This does not overly slow down our program, unless the memory becomes quite large. This causes the M to adjust based on the amount of N .

Space Complexity

Similar to the discussion over time complexity, the space complexity follows from the MaxHeap data structure, with the wrinkle of it being malloced. Generally, it is $O(n)$. But there is a lot that can be change here in terms of optimization, which I will discuss below:

Real Space Complexity

I will be using time with the `-l` flag (different than Linux's formatting, this is general UNIX) and find the *maximum resident set size* and interpret the result.

Sonnet's Real Space Usage

```
lucasfrias@Frias-iMac Fall2026_EECS348_Assignment_2 % /usr/bin/time -l  
./claud DEM0.txt
```

There are 3 emails to read.

There are 4 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important, Mind

Date: 01-03-2025

There are 2 emails to read.

0.06 real	0.05 user	0.01 sys
6803456	maximum resident set size	
0	average shared memory size	
0	average unshared data size	
0	average unshared stack size	
2770	page reclaims	
7	page faults	
0	swaps	
0	block input operations	
0	block output operations	
0	messages sent	
0	messages received	
0	signals received	
1	voluntary context switches	
7	involuntary context switches	
373897343	instructions retired	
217344675	cycles elapsed	

```
1425408 peak memory footprint
lucasfrias@Frias-iMac Fall2026_EECS348_Assignment_2 %
```

Compared to Luna's

```
lucasfrias@Frias-iMac Fall2026_EECS348_Assignment_2 % /usr/bin/time -l
./d_openai DEMO.txt
There are 3 emails to read.
There are 4 emails to read.
Next email:
Sender: Boss
Subject: Never Mind
Date: 01-03-2025
Next email:
Sender: Boss
Subject: Never Mind
Date: 01-03-2025
There are 3 emails to read.
      0.07 real      0.05 user      0.01 sys
      6811648 maximum resident set size
           0 average shared memory size
           0 average unshared data size
           0 average unshared stack size
      2771 page reclaims
        10 page faults
           0 swaps
           0 block input operations
           0 block output operations
           0 messages sent
           0 messages received
           0 signals received
           1 voluntary context switches
          18 involuntary context switches
    374084188 instructions retired
    219663422 cycles elapsed
      1433600 peak memory footprint
```

I believe the increase in space complexity from Luna is because of the struct sizing being considerably larger (see points below). Indeed, if we allocated 3000 elements to the email, which I did using this test file **generated by OpenAI's Luna mode**:

```
from datetime import date, timedelta
import random

random.seed(348)

senders = [
    "Boss",
    "ImportantPerson",
    "Peer",
    "OtherPerson",
]

subjects = [
    "Important",
    "Health Insurance Enrollment",
    "Never Mind",
    "Project Update",
    "Meeting Reminder",
    "Quarterly Review",
    "Schedule Change",
    "Action Required",
    "Team Lunch",
    "Budget Discussion",
    "Assignment Feedback",
    "Follow Up",
    "New Policy",
    "Important, Mind",
    "Please Review",
]

start_date = date(2024, 1, 1)
end_date = date(2025, 12, 31)
date_range = (end_date - start_date).days

with open("emails.txt", "w", encoding="utf-8") as file:
    for _ in range(3000):
        sender = random.choice(senders)
```

```

subject = random.choice(subjects)
email_date = start_date + timedelta(
    days=random.randint(0, date_range)
)

file.write(
    f"EMAIL {sender},{subject},"
    f"{email_date.strftime('%m-%d-%Y')}\n"
)

print("Created emails.txt with 3000 email records.")

```

I noticed that Luna can't handle 3,000 emails, while Sonnet can

```

Error: Maximum number of emails reached.
    0.08 real          0.05 user          0.01 sys
    7290880 maximum resident set size
         0 average shared memory size
         0 average unshared data size
         0 average unshared stack size
    2905 page reclaims
         7 page faults
         0 swaps
         0 block input operations
         0 block output operations
         0 messages sent
         0 messages received
         0 signals received
        20 voluntary context switches
        19 involuntary context switches
    397836697 instructions retired
    233561459 cycles elapsed
    1904640 peak memory footprint

```

Generally, by adjusting the range, I found that Luna used around 10,000 bytes more on average than Sonnet. See findings below to make sense of this:

Struct Padding

You might already know where I'm going with this, but let's look at Luna and Sonnet's structs, their layout and then discuss:

Luna's Email Struct

```
#define MAX_CATEGORY 30
#define MAX_SUBJECT 500
typedef struct
{
    char sender[MAX_CATEGORY];
    char subject[MAX_SUBJECT];
    char date[20];

    int categoryPriority;
    int dateValue;
} Email;
```

Sonnet's Email Struct

```
#define MAX_CATEGORY_LEN 32
#define MAX_SUBJECT_LEN 256

typedef struct {
    char category[MAX_CATEGORY_LEN]; /* Boss, Subordinate, Peer, ...
    */
    char subject[MAX_SUBJECT_LEN]; /* subject line
    */
    char date[11]; /* MM-DD-YYYY (10 chars + NUL)
    */
    int categoryRank; /* numeric priority of category
    */
    long dateValue; /* YYYYMMDD, for easy comparison
    */
} Email;
```

Okay, first and probably most significant detail is the long versus int on the datevalue. Don't really know why Sonnet chose long here. An int represents (unless we're on really old C) 4 bytes, and 2^{32} is way more than enough space to represent millenia to come. Long, however, is much larger and very unnecessary. It makes larger Emails, which are totally unnecessary.

Sonnet does recognize that we only need 11 characters to represent the date, however, which gives its struct less size. When we build our own custom types, we want to implement the smallest byte size, especially when we have something that occurs a lot (like emails).

To also note, both Sonnet and Luna have ridiculously long email subject permissions. According to Mr. Petrotta from StackOverflow:

(<https://stackoverflow.com/questions/1592291/what-is-the-email-subject-length-limit>)

See RFC 2822, section 2.1.1 to start.

There are two limits that this standard places on the number of characters in a line. Each line of characters MUST be no more than 998 characters, and SHOULD be no more than 78 characters, excluding the CRLF.

He continues to say:

The recommendation for no more than 78 characters in the subject header sounds reasonable. No one wants to scroll to see the entire subject line, and something important might get cut off on the right.

Testing this on a modern email client (this post is from 2009, after all) it seems like KU's outlook has a limit of 256. This seems reasonable. 500 is too much, and makes every email twice as expensive. Less is even possible, although maybe not good to implement.

I made a quick little nonsense C program to check this:

```

// Online C compiler to run C program online
#include <stdio.h>
/* Type your code here, or load an example. */
#define MAX_CATEGORY_LEN 32
#define MAX_SUBJECT_LEN 256
#define MAX_CATEGORY 30
#define MAX_SUBJECT 500

int main() {
    // Write C code here
    typedef struct
    {
        char sender[MAX_CATEGORY];
        char subject[MAX_SUBJECT];
        char date[20];

        int categoryPriority;
        int dateValue;
    } Email;

    typedef struct {
        char category[MAX_CATEGORY_LEN]; /* Boss, Subordinate, Peer, ...
        */
        char subject[MAX_SUBJECT_LEN]; /* subject line
        */
        char date[11]; /* MM-DD-YYYY (10 chars + NUL)
        */
        int categoryRank; /* numeric priority of category
        */
        long dateValue; /* YYYYMMDD, for easy comparison
        */
    } Email2;

    Email rest = {0,0,0,0,0};
    Email2 test = {0,0,0,0,0};
    printf("Sonnet is: %d\n", sizeof(test));
    printf("Luna is: %d", sizeof(rest));

```

```
    return 0;
}
```

And the result:

```
Sonnet is: 312
Luna is: 560
```

shows that Luna's is almost 80% more memory than Sonnet's.

Perhaps, the most important thing: struct ordering.

To refresh, when creating a struct in C, all the memory is contiguous. Because of how C structs operate, they need a certain amount of spacing for the memory. For example:

```
typedef struct {
    int age
    char letter
} Guy;
```

We just made some Guy, and we would expect the size of this struct to be 4 + 1 to be five. However, actually getting sizeof, we get 8. Why?

C will try and set the offsets of the table of accessible addresses the same width, generally speaking, for alignment. This can be platform dependent, but this example should generally hold true for almost all C compilers. It is strange because this changes depending on exactly how the struct is ordered. For example:

```
typedef struct {
    int age
    char letter
    char smetter
    char better
    char netter
} Guy;
```

Also has 8 bytes of size, because the spacing is given for one char byte for each. It is a very strange result of the inner workings of C.

Struct alignment, when done right, saves a lot of space. The long's addition in Sonnet makes the padding larger just where it is itself, but the categoryRank can be a uint8_t (it doesn't have that much range). I suspect the AI is generating older C, again, and doesn't know about 1 byte types.

Recommended changes by me would definitely be to make a shorter subject line (let's say 256, which Sonnet does) and then also to implement smaller representations of the data. This is where I think a lot of problems with spacing and how many emails can be malloced onto the heap. Something like

```
typedef struct {  
    uint8_t categoryType;  
    uint8_t senderType;  
    int32_t dateValue;  
    char subject[MAX_SUBJECT_LEN];  
}
```

would be much more efficient, less verbose, and frankly better.

Heap Space Complexity Benefits

A brief statement is included here to laud the choice of using a heap allocated array instead of a stack array. While slower, it allows for significantly more emails and has much less potential to cause a stack overflow or other UB resulting from large amounts of data being processed.

My criticism in Heap vs Stack in Correctness is just to note that humans would, generally, not program a heap allocation method, which makes it seem alien. While reviewing this code more, it makes me appreciate the implementation of using a heap based array for a MaxHeap with considerably large elements, even if it is more difficult to implement.

String Storing

I touched on this in the struct, but I would like to revisit the string storing in structs and how unnecessary it is and increases string sizing.

To start with the date, storing both the integer and string representation is wasteful, for every single email. In a program like this where space matters, that is 11 extra bytes for every single struct, best case scenario. Luna's is actually 20, for no specific reason. If the dateValue can be represented and displayed as an integer, there is no reason to not implement a function to just get the dateValue info. Strings are harder to work with than integers and contain much more potential problems.

The only design reason to include the string is, if the string is constantly used by other parts of the program, it would be annoying to generate a string for it and then free it, because it would get difficult. But in the scope of this program, where we only need to display the date formatted from the integer, this point is moot and not really useful.

Also, because of the nature of this problem, we can represent the sender as a value in the range of a value between 0-4 (Boss, Subordinate, Peer, Important Person, Other Person) and using strcmp for the char * value seems wasteful and long, since the problem statement only has 5 statements. If this wasn't a "toy" problem, we of course would want to store the string value, but in the PROMPT.md given to the LLMs, the senders were specified to be finite values.

An enum would also be a great choice here, although the reason I prefer using a uint8_t is because the namespace can be implemented using macros and the enum would be 32-bit unless you typedefine the macro, which was implemented in C23 (which is a bit too modern to rely on). Still, uint8_t < enum < char [namesize] in terms of space.

Fixing the string storing in the struct is the biggest optimization in space that can be done to these structs.

Maintainability

As much as these are impressive to generate in one prompt, with no style guide, it is very clear that the choices made by the large language models are not great.

Scalability

To give both Luna and Sonnet the appropriate lauding, it is better to process a large amount of data through the heap instead of the stack. This allows them to store a greater amount of data, and makes for a more robust system. Also robust in the implementation of string fields for senders for multiple different senders.

The LLMs definitely understand that critical infrastructure like emails ought to be scalable. Maybe this is some incentive they have, built to try and implement the most durable code, especially in C.

Problem Statement Optimizations

However, as Donald Knuth once said: "Premature optimization is the root of all evil". The LLMs, implementing the most general solution possible, fail to make common sense choices in their implementations. For example, there are only five possible senders in the PROMPT.md specification. There is no need to store the string (as discussed in Space Complexity->String Storing). Moreover, the choice to use the heap choice may be more difficult to actually maintain in the future (from Correctness->Heap vs Stack). I feel that, even though this is optimized for working in the long term, it is too general of an implementation that would definitely be questioned at any sort of interview.

The solution to this problem does not require some of these design choices, and actively make maintaining this with a human worse.

Double Storing Date

Touched in on struct but this will generate confusion specifically with maintainability and actually configuring the values and outputs. Some future feature may attempt to use one over the other, and if something changes (for example, leap year calculations) it is easier to work with the integer than string version. There shouldn't be a reason to modify the date, but it should remain one value.

Older C89 Syntax

I believe this may become a common theme, but Luna and Sonnet are trained off of older C information and data. Here are some examples:

```
//From Sonnet's heapSiftDown function:
while (1) { //no stdbool
    int left  = leftChildIndex(index);
    int right = rightChildIndex(index);
    int largest = index;
    ...
}
```

Several integer return values instead of boolean, which is a little boring but true, there is no good reason to use an int:

```
//From Sonnet's heapPeek:  
int heapPeek(MaxHeap *heap, Email *outEmail) {  
    if (heap->size == 0) return 0;  
    *outEmail = heap->data[0];  
    return 1;  
}
```

But generally not as bad as it could possible be. Sonnet even used size_t!

```
//From Sonnet's trimNewline:  
  
void trimNewline(char *s) {  
    size_t len = strlen(s);  
    while (len > 0 && (s[len - 1] == '\n' || s[len - 1] == '\r')) {
```

General Thoughts

Despite my criticism, this is not unmaintainable. It just is too general for the problem statement, which does help it in maintainability. Even still, it makes choices that would make it harder to modify for future additions and might cause confusion. This is the center of my criticism: LLMs, trained on general data, seem very good at making general choices, but struggle when it comes to common-sense problem statement optimizations.

Human Output

After discussing and figuring out both, I prefer Sonnet's output. It is more error reducing than Luna's, and I believe that the structure and clarity of Sonnet's output are superior in this application so far.

If I had the chance to program this from the ground up, I would use a stack MaxHeap. However, I do not want to overly modify Sonnet's code, so I will implement just some of the struct optimizations, namely the enum and the integer date.

Human Modified Output (From Sonnet)

```

/**
 *  <@)
 *    KU//
 *    " "
 *  KU 2026 -- make cool stuff!!
 *  by lucas frias
 *
 *  human.c - EECS 348 Assignment 2
 *  Description: Prioritizes a busy CEO's inbox using a MaxHeap
(array/list based
 *  priority queue) implemented entirely from scratch
 *  Output: outputs specified in the PROMPT.md based on the rubric and
instructions
 *
 *  Author: Lucas Frias
 *  Collaborators: Lucas, base implementation from Sonnet, debugging from
Sonnet,
 *  ZedAI autocomplete for some helper enum functions (notated below)
 *  Created: September 4th 2026
 *  Revised September 16th 2026
 *
 *
 *  the rest is sonnet's prompt, i'll add it here for reasoning purposes
 *  =====
 *  email_priority.c
 *
 *  Prioritizes a busy CEO's inbox using a MaxHeap (array/list based
 *  priority queue) implemented entirely from scratch -- no library
 *  heap functions are used.
 *
 *  Commands read from a script file (given as argv[1]) or stdin:
 *
 *    EMAIL <sender category>,<subject line>,<date MM-DD-YYYY>
 *    NEXT
 *    READ
 *    COUNT

```

```

*
* Sender category priority (highest to lowest):
*   Boss > Subordinate > Peer > ImportantPerson > OtherPerson
*
* Within the same category, the NEWEST email (largest date) is
* read first.
* ===== */

#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <stdbool.h>

/* -----
* Constants
* ----- */
//MAX_SUBJECT_LEN is the maximum length of a subject line
//INITIAL_CAPACITY is the initial capacity of the heap, can grow later
//GENERATED BY SONNET
#define MAX_SUBJECT_LEN    256
#define INITIAL_CAPACITY   16

//define sender ahead of time here
enum Sender {
    //we have to do these in reverse order so that boss is a higher
    number
    // enums start at 0, so boss is the highest value and we can use
    this to compare
    SenderUnknown,
    SenderOtherPerson,
    SenderImportantPerson,
    SenderPeer,
    SenderSubordinate,
    SenderBoss,
};

```

```

/* -----
 * Email struct -- GENERATED BY SONNET, modified by Lucas
 * ----- */
typedef struct {
    char subject[MAX_SUBJECT_LEN];    /* subject line
*/
    enum Sender categoryRank;          /* enumerate priority of
category */
    uint32_t dateValue;                /* YYYYMMDD, for easy
comparison */
} Email;

/* -----
 * MaxHeap struct (list/array based implementation) -- GENERATED BY
SONNET
 * ----- */
typedef struct {
    Email *data;    /* dynamically allocated array of emails */
    int    size;    /* number of elements currently in heap */
    int    capacity; /* allocated capacity of the array      */
} MaxHeap;

/* =====
 * Category priority lookup
 *
 * Higher number == higher priority (read sooner).
 * ===== */

//ADDED BY LUCAS, changed from Sonnet's same function, used ZedAI
autocomplete
enum Sender categoryRank(const char *category) {
    //categoryRank -> given a category string passed to us, we are able
to
    // return the corresponding sender rank by using strcmp to compare
the category

```

```

    // and returns the corresponding enum rank
    // if the category is not recognized, returns SenderUnknown, so
super low
    // priority. also strcmp is safer because strncmp requires a length
parameter
    // we're just checking for an exact match, so strcmp is fine
    if (strcmp(category, "Boss") == 0)          return SenderBoss;
    if (strcmp(category, "Subordinate") == 0)    return
SenderSubordinate;
    if (strcmp(category, "Peer") == 0)          return SenderPeer;
    if (strcmp(category, "ImportantPerson") == 0) return
SenderImportantPerson;
    if (strcmp(category, "OtherPerson") == 0)    return
SenderOtherPerson;
    return SenderUnknown;
}

///ADDED BY LUCAS -> function to print the date. used ZedAI autocomplete
void printDate(uint32_t date) {
    //printDate -> given a uint32_t date value that represents a date
    // in just a number instead of bits, prints it as a formatted thing
    // e.g. 20240901 -> 09-01-2024
    // doesn't return anything, just prints to stdout

    //year, month, day we do some quick calculations
    // year we can integer divide to get
    int year = date / 10000;
    // month we can get by taking the remainder after dividing by 10000,
then dividing by 100
    int month = (date % 10000) / 100;
    // day we can get by taking the remainder after dividing by 100
    int day = date % 100;
    //print it with the correct integer spacing
    printf("%02d-%02d-%04d", month, day, year);
}

```

```
//ADDED by Lucas - better printing for sender from enum - used ZedAI
autocomplete
void printSender(enum Sender sender) {
    //printSender -> given an enum Sender value, prints the sender as a
string
    // doesn't return anything, just prints the thingymajig
    switch (sender) {
        /*i'm gonna explain this so i don't gotta comment every
        * line because that's gotta be excessive, we just
        * basically match the enum value, and then when we get it
        * we can just printf for a value. we know that the enum
        * has to be within this range because we guarantee it
        * when we generate our categoryRank, so we don't need
        * a default case for this. we print using printf, and then
        * we break so we don't fall through to the next case
        */
        case SenderUnknown:
            printf("Unknown");
            break;
        case SenderBoss:
            printf("Boss");
            break;
        case SenderSubordinate:
            printf("Subordinate");
            break;
        case SenderPeer:
            printf("Peer");
            break;
        case SenderImportantPerson:
            printf("ImportantPerson");
            break;
        case SenderOtherPerson:
            printf("OtherPerson");
            break;
    }
    printf("\n");
}
```

```

/* =====
 * Convert MM-DD-YYYY into a comparable integer YYYYMMDD.
 * Larger value == newer date.
 * ===== */

//GENERATED BY SONNET
uint32_t dateToValue(const char *date) {
    //dateToValue - converts a date string in MM-DD-YYYY format to a
    comparable integer YYYYMMDD
    // returns it as a uint32_t to make sure we have enough space bc
    windows
    // sometimes does long as 4 bytes how scandalous
    //sonnet does a fancy int initialize in oneline
    int month = 0, day = 0, year = 0;
    //sscanf will try to cast the value from the date into the addresss
    // into like a reverse printf statement. if it returns 3, then we
    // know all three addresses got written to, but if it doesn't
    // sonnet kinda throws in the towel
    if (sscanf(date, "%d-%d-%d", &month, &day, &year) != 3) {
        return 0; /* malformed date, treat as very old */
    }
    //returns a uint32_t value representing the date in YYYYMMDD format
    // like how we said, we just added all of the values together up
    return ((uint32_t)year * 10000) + (month * 100) + day;
}

/* =====
 * Compare two emails for max-heap ordering.
 * Returns 1 if "a" has strictly higher priority than "b", else 0.
 *
 * Priority rules:
 * 1) Higher categoryRank wins.
 * 2) If categories tie, newer date (larger dateValue) wins.
 * ===== */
//generated completely by Sonnet except Lucas turned it from int to bool
bool emailHasHigherPriority(const Email *a, const Email *b) {

```

```

    //bool emailHasHigherPriority - rates emails on the
    // priority rules defined in the assignment spec
    // takes two email pointers and returns true if "a" has higher
priority than "b"
    if (a->categoryRank != b->categoryRank) { //check to see if they're
not the same thing
        //oh cool they're not, that means we can just return whichever
one has the greater category rank
        // btw this works like how the comp void ptr function works for
        // qsort in the ANSI C library where it returns a boolean for
the sorting
        // just so happens that there are only two possibilities
        return a->categoryRank > b->categoryRank;
    }
    //now we just return the date value comparison if a < b
    return a->dateValue > b->dateValue;
}

/* =====
* MaxHeap creation / destruction
* ===== */

//generated by sonnet
MaxHeap *heapCreate(unsigned int initialCapacity) {
    //heapCreate - allocates and initializes a MaxHeap with the given
capacity
    //returns a pointer to the newly created heap, or exits if memory
allocation fails
    // initialCapacity: the initial capacity of the heap, although
    // i changed it to be an unsigned int because that makes more sense
    //
    //mild syntax edit, Sonnet did a recast even though it's a void
pointer
    MaxHeap *heap = malloc(sizeof(MaxHeap));
    //okay now we check if we failed to make a heap. malloc will
    // return a pointer to 0 if it fails, so if we do !0 it will be
    // true

```

```

    if (!heap) {
        //now we freak out and exit with exit
        fprintf(stderr, "Fatal: out of memory creating heap.\n");
        exit(EXIT_FAILURE);
    }
    //now we allocate the backing array for the heap
    // i also removed the explicit cast to Email * again
    // more pedantic stuff
    heap->data = malloc(sizeof(Email) * initialCapacity);
    //same nullcheck we did for the heap pointer before
    if (!heap->data) {
        //same error handling we did for the heap pointer before
        // same old same old folks
        fprintf(stderr, "Fatal: out of memory allocating heap
storage.\n");
        exit(EXIT_FAILURE);
    }
    //we need to remove the garbage stuff and explicitly initialize
everything
    heap->size = 0; //no elements yet set to zero
    heap->capacity = initialCapacity; //capacity is the default initial
capacity
    return heap; //returns the heap pointer to the caller
}

//generated by Sonnet, slightly modified by Lucas
void heapDestroy(MaxHeap *heap) {
    //heapDestroy, given a heap pointer, free that guy
    // by first removing his data and then the heap itself
    if (!heap) return; //make sure the heap pointer is not null
    //and we're not dereferencing a null pointer
    if(!(heap->data)) return; //same thing here, this is ADDED BY LUCAS
    //sonnet never checked the data, but if we're checking the heap
    // pointer, we should check the data too, right?
    free(heap->data); //free the data first
    free(heap); //then the heap pointer, because otherwise we would have
no pointer to free

```

```

}

/* Double the backing array when it fills up. */
//generated by Sonnet,
void heapEnsureCapacity(MaxHeap *heap) {
    //heapEnsureCapacity, given a heap pointer, double the capacity of
the heap
    // very, very carefully
    // first check if the heap needs to be expanded this
    // function gets called every add which, is actually
    // really inefficient, but whatever guys
    if (heap->size < heap->capacity) return;
    //okay now we double the capacity value. we actually don't check
    // for integer overflow here, but if the capacity is too large,
    // but whatever right, it would be hard to exceed that (maybe)
    int newCapacity = heap->capacity * 2;
    //get the newData pointer, which is the reallocate memory block. We
have
    // to do this explicitly to get the same contiguous block of memory
    // that we started with, so we can free it later without issues
    // i also modified the realloc call to use the same type as the
original
    // data pointer, so we don't have to cast it back to Email* later
    Email *newData = realloc(heap->data, sizeof(Email) * newCapacity);
    //check if the realloc call succeeded, and exit if it didn't
    // it's just making sure we don't have a null pointer
    if (!newData) {
        //exit out and whine to the user
        fprintf(stderr, "Fatal: out of memory growing heap.\n");
        exit(EXIT_FAILURE);
    }
    //set the data pointer to the new memory blocks, nothing fancy here
    heap->data = newData;
    heap->capacity = newCapacity;
}

/* -----

```

```

* Index helpers for the array-based (list-based) heap layout.
* For node at index i (0-based):
*   parent  = (i - 1) / 2
*   left    = 2*i + 1
*   right   = 2*i + 2
* ----- */

//all of these functions are generated by sonnet, but i added inline
// to improve performance by avoiding function call overhead.
// makes it a tiny bit faster
inline static int parentIndex(int i) { return (i - 1) / 2; }
inline static int leftChildIndex(int i) { return 2 * i + 1; }
inline static int rightChildIndex(int i) { return 2 * i + 2; }

//this function is pure sonnet
static void swapEmails(Email *a, Email *b) {
    //just swaps the contents of two Email pointers,
    //no need to copy the entire struct
    Email temp = *a;
    *a = *b;
    *b = temp;
}

/* -----
* Sift-up (bubble-up): restores heap property after insertion
* at the end of the array.
* ----- */

//this is sonnet
void heapSiftUp(MaxHeap *heap, int index) {
    //heapSiftUp - given a maxheap and an index where we start,
    //bubble up the element to its correct position
    while (index > 0) { //because we would stop at zero right
        int parent = parentIndex(index); //get the parent index, the
higher node
        //check the priority based on our data

```

```

        if (emailHasHigherPriority(&heap->data[index], &heap-
>data[parent])) {
            //is out dada higher than us? then swap
            // and we become the alpha (we are the parent
            // and they are our child)
            swapEmails(&heap->data[index], &heap->data[parent]);
            //and keep bubbling up
            index = parent;
        } else {
            //we're done! we can stop sifting up
            break;
        }
    }
}

/* -----
 * Sift-down (bubble-down): restores heap property after the
 * root is replaced (e.g. during extraction).
 * ----- */

//generated by sonnet
void heapSiftDown(MaxHeap *heap, int index) {
    //heapSiftDown - given a maxheap pointer
    // and an index where we star
    //bubble down the element to its correct position
    // this happens after a pop and we need to re-order

    //keep bubbling down until we find the correct position
    while (true) {

        //find the largest of the current node and its children
        //first get our left and right
        int left  = leftChildIndex(index);
        int right = rightChildIndex(index);
        //then get our index, set it to the targets
        int largest = index;
    }
}

```

```

        //if the largest is not the current node, swap and continue
        if (left < heap->size &&
            emailHasHigherPriority(&heap->data[left], &heap-
>data[largest])) {
            largest = left;
        }
        //do the same but for the right, we change our stuff
        if (right < heap->size &&
            emailHasHigherPriority(&heap->data[right], &heap-
>data[largest])) {
            largest = right;
        }
        //if we neither need to go left or right, then we break
        // because we are now reorganized
        if (largest == index) break;
        //swap with the largest child because we found a new largest
value
        swapEmails(&heap->data[index], &heap->data[largest]);
        //set the index to be the largest
        index = largest;
    }
}

/* -----
 * Insert a new email into the heap.
 * -----
 * heapInsert is generated by Sonnet*/
void heapInsert(MaxHeap *heap, Email email) {
    //heapInsert, given a maxheap pointer and an email,
    //just throw it into the heap at the i+heapsize position and
    // then heapShift it up to the correct position starting
    // from it's i+heapsize position

    heapEnsureCapacity(heap); //first make sure we have enough space
    heap->data[heap->size] = email; //second set the email at the new
position
    heapSiftUp(heap, heap->size); //now we sift all the way up

```

```

    heap->size++; //we increment the size because we added a new element
    and for the next insert
}

/* -----
 * Peek at the highest-priority email without removing it.
 * Returns 1 on success, 0 if heap is empty.
 * ----- */

//generated by Sonnet, bool by Lucas
bool heapPeek(MaxHeap *heap, Email *outEmail) {
    //heapPeek, given a maxheap ptr and a email ptr, return a boolean
    // of the success at writing the out email
    if (heap->size == 0) return false; //we can't peek nothing
    *outEmail = heap->data[0]; //set the value
    return true; //return a boolean value that says yay!! we did it
}

/* -----
 * Remove and return the highest-priority email.
 * Returns 1 on success, 0 if heap is empty.
 * ----- */

//generated by Sonnet, bool by Lucas
bool heapExtractMax(MaxHeap *heap, Email *outEmail) {
    //heapExtractMax, given a maxheap ptr and a email ptr, return a
    boolean
    // of the success of us getting the Max value (the top). we
    // write it to outEmail and then remove it from the heap
    // because of the read thingamajigger
    if (heap->size == 0) return false;

    //extract the max value (the top) and write it to outEmail
    *outEmail = heap->data[0];
    //deincrement our heap
    heap->size--;

    if (heap->size > 0) { //make sure we don't try and sift down

```

```

        //a zero sized heap
        //now we set the top to the last element in the heap
        heap->data[0] = heap->data[heap->size];
        //we need to resift down and compare every index
        // and make sure they preserve the order
        heapSiftDown(heap, 0); //starting at our new head
    }
    return true;
}

//generated by sonnet, made a tiny bit safer by lucas
int heapSize(MaxHeap *heap) {
    //given a heap ptr, return the size of the heap
    if (!heap) return 0; //return 0 for an uninit heap (nullptr)
    return heap->size; //return the heap size
}

/* =====
 * String helpers
 * ===== */

/* Strip trailing newline / carriage return characters. */
//generated by sonnet,
void trimNewline(char *s) {
    //trimNewline, given a char * s, let's remove every
    // single newline and tab or whatever other formatting nonsense
    //first, let's get the len of the str
    size_t len = strlen(s);
    //now, let's loop through the str, removing any trailing newlines
    // basically, while our len is greater than 0 and our last char is
    // a newline or tab character
    // we remove it and decrement our len by one.
    // this is only for the end of the line, very specifically
    // for the sender/subject
    while (len > 0 && (s[len - 1] == '\n' || s[len - 1] == '\r')) {
        s[len - 1] = '\0';
        len--;
    }
}

```

```

    }
    //the pointer is the one being worked on, so we don't have to do
    anything
}

/* Remove leading/trailing whitespace in place. */
//generated by sonnet, improved by lucas
void trimWhitespace(char *s) {
    //trimWhitespace, given a char * s, deletes
    // all spacin characters in a confusing way
    /* Trim trailing */
    //fixed by lucas to use size_t, but get the strlen
    size_t len = (size_t)strlen(s);
    //now we loop while we still have a len, and we check if
    // the char is a space with isspace from the part
    //this part removes the trailing (end part) whitespace
    while (len > 0 && isspace((unsigned char)s[len - 1])) {
        s[--len] = '\0';
    }
    /* Trim leading */
    //^^ yeah what sonnet said, but we keep track and
    // then memcpy later you'll see
    int start = 0;
    while (s[start] != '\0' && isspace((unsigned char)s[start])) {
        //loop while we still have a char and it's a space, add for
start
        start++;
    }
    if (start > 0) {
        //kinda recopy the string over itself from the start
        // before there's any spaces, pretty sick
        memmove(s, s + start, strlen(s + start) + 1);
    }
}

/* =====
* Parse an "EMAIL <category>,<subject>,<date>" line (the text

```

```

* AFTER the leading "EMAIL " token has already been stripped
* off by the caller) into an Email struct.
*
* Returns 1 on success, 0 on a malformed line.
* ===== */
//generated by sonnet, fixed a bit by lucas
bool parseEmailFields(char *fields, Email *outEmail) {
    //parseEmailFields, given a char * to the email field statement
    // generated an Email and return it
    /* fields looks like: "Boss,Never Mind,01-03-2025" */
    //first let's split with the first comma
    char *firstComma = strchr(fields, ',');
    //if this can't happen, the line is malformed return
    if (!firstComma) return false;
    //we set the split part, instead of being a comma, to
    // a null term so we can reinterpret it as a value later
    // we basically mutate the string with a bunch of
    // null terms and then read from the ptrs so we get
    // Boss \oNever Mind\0 01-03-2025\0 or something like that
    *firstComma = '\0';
    char *category = fields;//set category to the first part, this is
    //just the beginning base pointer
    char *rest = firstComma + 1;//we do pointer arithmetic for the
    //second part of the string
    char *secondComma = strrchr(rest, ','); /* subject can't have
commas,
so the LAST comma in the
remainder separates the
subject from the date */
    if (!secondComma) return false;//same error check as before
    *secondComma = '\0';//set the second part to a null term so we can
    //read it as a value later
    char *subject = rest;//set our pointer to the start after the first
    null term
    char *date = secondComma + 1;//the date is the rest
    //we don't need to parse again, this is the last field and it is
    already

```

```

// null terminated

//remove all the whitespace from the fields, passing
// our pointer and letting them do work
trimWhitespace(category);
trimWhitespace(subject);
trimWhitespace(date);

//check for malformed or empty fields and return a false
//if one or the other is empty, the line is malformed
if (strlen(category) == 0 || strlen(date) == 0) return false;

//okay now we can safely parse the fields, first we get the category
// rank as an enum value
outEmail->categoryRank = categoryRank(category);

//now we copy the subject into the email struct, making sure to
// null terminate it and do truncation if necessary when it
// exceeds the maximum subject length
strncpy(outEmail->subject, subject, MAX_SUBJECT_LEN - 1);
//add that pesky null term character to the end, just in case
outEmail->subject[MAX_SUBJECT_LEN - 1] = '\0';
//set the date value by calling dateToValue on the date string,
//sets the uint32_t date value for easy comparison
outEmail->dateValue = dateToValue(date);
return true;//return boolean signaling we did it
}

/* =====
 * Command handlers
 * ===== */
//generated by sonnet
void handleEmailCommand(MaxHeap *heap, char *line) {
    //handleEmailCommand: given a heap ptr, and an EMAIL line,
    interprets
    // what to do with the values given from each line

```

```

/* line begins with "EMAIL ", the rest is the field data */
char *fields = line + strlen("EMAIL");
//fixes formatting ehre
while (*fields == ' ') fields++; /* skip the single space (and any
extra) */

Email email;//make a new email here
//if we return a false value, meaning we couldn't parse the fields,
//we print an error message and return
if (!parseEmailFields(fields, &email)) {
    //throw an error
    fprintf(stderr, "Warning: could not parse EMAIL line: %s\n",
line);
    return;
}
//insert our email into the heap, pass by value
heapInsert(heap, email);
}

void handleNextCommand(MaxHeap *heap) {
    //handleNextCommand: given a heap ptr, prints the next email in the
heap
    //if there are no emails, prints a message and returns
    Email top;
    //if we have false, that means our heap is of size 0
    if (!heapPeek(heap, &top)) {
        printf("Next email:\n");
        printf("There are no emails to read.\n\n");
        //premature return to avoid next steps
        return;
    }
    //alright now we have an email to print
    printf("Next email:\n");
    printf("Sender: ");
    //display the sender as a string from the enum value
    printSender(top.categoryRank);
    //print the subject, it's just a string so we use %s

```

```

    printf("Subject: %s\n", top.subject);
    //print the date, it's a uint32_t reverse formatted
    // so we use that special printDate
    printf("Date: ");
    printDate(top.dateValue);
    printf("\n\n");
}

void handleReadCommand(MaxHeap *heap) {
    //handleReadCommand, given a maxheap *, removes the top email and
    prints it
    // first we check if there's anything to remove
    Email dequeued;
    //check to make sure there's something to remove
    // heapExtractMax will also print the info if it
    // exists, so not handled here
    // actually don't know why we have the dequeued thing
    // to begin with it's kinda pointless except for having
    // some address to write to
    if (!heapExtractMax(heap, &dequeued)) {
        printf("There is nothing to mark as read.\n\n");
    }
    /* Per spec, READ simply removes the current top email
       (the CEO dealt with it). No output is required. */
    //this is sonnets note here, and it's true ^^
}

void handleCountCommand(MaxHeap *heap) {
    //handleCountCommand, given a maxheap *, prints the number of emails
    to read
    //we just print the integer version of the heap size
    printf("There are %d emails to read.\n\n", heapSize(heap));
}

/* =====
* Main driver: reads commands line by line from the given file

```

```

* (or stdin if no file argument is provided) and dispatches
* them to the appropriate handler.
* ===== */

//main, generated by sonnet
int main(int argc, char *argv[]) {
    //main, given a file argument, reads commands from the file
    // and sends those guys to the appropriate handler
    FILE *input = stdin;//get the file from the command line

    //make sure we have enough arguments
    //i modified this to make sure it's the one file
    // because that's the only support we have
    if (argc == 2) {
        //now let's try and open the file
        input = fopen(argv[1], "r");
        //if the input is null, we couldn't open the file (nullptr)
        if (!input) {
            //freak out!!
            fprintf(stderr, "Error: could not open file '%s'\n",
argv[1]);
            return EXIT_FAILURE;
        }
    }
    else{printf("Error! Please include a file\n"); return EXIT_FAILURE;}

    //now let's create the heap, using our initial capacity macro
    MaxHeap *heap = heapCreate(INITIAL_CAPACITY);

    char line[1024];//allocate space for the line to interpret
    //fgets reads a line from the file into the buffer
    while (fgets(line, sizeof(line), input)) {
        //remove the newline stuff
        trimNewline(line);
        //added by lucas inspired by sonnet's past error checking
        // seems smart to make usre
        line[1023] = '\0';
    }
}

```

```

/* Skip blank lines */
char *check = line;
//skip leading whitespace on the thing, by incrementing
// our check pointer by 1 if we have them
while (*check == ' ' || *check == '\t') check++;
if (*check == '\0') continue;//break out of this
//iteration because it's an empty line

/* big comment here,
 * okay so we have a line, let's check what it is
 * we incremented all the way to the point where
 * there is no longer any whitespace, so now we
 * have a non-empty line to work with. it might
 * be an EMAIL, NEXT, READ, or COUNT command
 * or it might not be at all. the way sonnet validates
 * this is by just check the first N characters,
 * where N is the length of the command string that
 * could or could not be there. this avoids the
 * error with strcmp that Luna had with some weird
 * whitespace characters at the end of the line
 * that would mess up its jam. we just hand
 * it off to its appropriate helper command.
 * 0 means no error, and then i'll describe what
 * each helper command does.
 */
if (strncmp(line, "EMAIL", 5) == 0) {
    //adds an email to the heap
    handleEmailCommand(heap, line);
} else if (strncmp(line, "NEXT", 4) == 0) {
    //prints the next email in the heap
    handleNextCommand(heap);
} else if (strncmp(line, "READ", 4) == 0) {
    //removes the top email and prints it
    handleReadCommand(heap);
} else if (strncmp(line, "COUNT", 5) == 0) {
    //prints the number of emails left to read

```

```

        handleCountCommand(heap);
    } else {
        //prints an error message for unrecognized commands
        fprintf(stderr, "Warning: unrecognized command: %s\n",
line);
    }
}

if (input != stdin) {
    //close our input file
    fclose(input);
}
//free the heap and all its memory
heapDestroy(heap);
//we did it!! return success
return EXIT_SUCCESS;
}

```

This implementation didn't change much because I think a structural rewrite into a stack-based MaxHeap would be better for this particular application, but the work and difference between the AI code would be so significant that it would be beyond the scope of adding small improvements. That being said, I don't think this is particularly bad of a solution, now that the structs are smaller. Sonnet did a good job anticipating output issues.

Improved "Correctness" and Maintainability

Sonnet's original code was correct. I changed some minor syntax (for example, explicitly recasting a malloced pointer twice like this:)

```
int * doobyDoo = (int *)malloc(sizeof(int));
```

and adding `true` and `false` and less weird syntax overall. These changes are discussed in the comments attributed to Lucas modification.

These changes are what makes code maintainable. Using weird syntax and implicit features of the C language make code less readable for most people, and Sonnet made a lot of programming choices that are bizarre to read until you fully wrap your head around. By

changing some of this to more standard syntax, the overhead for maintainability decreases sharply.

Improved Space and Time Complexity Resulting From Struct Optimization

I used Claude Code (Opus High) to do a space and time complexity difference in each implementation. Opus generated this test file in my tmp folder:

```
awk 'BEGIN{
    srand(42);
    cats[0]="Boss"; cats[1]="Subordinate"; cats[2]="Peer";
    cats[3]="ImportantPerson"; cats[4]="OtherPerson";
    n=500000;
    for(i=0;i<n;i++){
        c=cats[int(rand()*5)];
        mm=int(rand()*12)+1; dd=int(rand()*28)+1; yy=2024+int(rand()*2);
        printf "EMAIL %s,Subject line number %d,%02d-%02d-%04d\n", c, i, mm,
        dd, yy;
    }
    for(i=0;i<n;i++) print "READ";
}' > /tmp/bench_big.txt
```

And ran three iterations of each implementation on this test file, which is pretty good for determining whether the optimization matters on cache locality

```
echo "=== Baseline (anthropic.c) ==="
for i in 1 2 3; do /usr/bin/time -p /tmp/bench_anthropic
/tmp/bench_big.txt > /dev/null; done

echo "=== Modified (human.c) ==="
for i in 1 2 3; do /usr/bin/time -p /tmp/bench_human /tmp/bench_big.txt
> /dev/null; done
```

And also did a memory comparison using `time -l`

```
/usr/bin/time -l /tmp/bench_anthropic /tmp/bench_big.txt > /dev/null
/usr/bin/time -l /tmp/bench_human /tmp/bench_big.txt > /dev/null
```

I independently looked at, verified, and ran these commands. Here's the output:

- Sonnet used 312 bytes as opposed to my 264 bytes
- The modified execution time was on average, decently faster than the baseline:

	iter 1	iter 2	iter 3	average
anthropic.c	1.23 s	1.17 s	1.16 s	1.19 s
human.c	1.10 s	1.08 s	1.09 s	1.09 s

- With my basic optimizations, there was nearly a 9% decrease on CPU time
- Memory was a bit more. Given a 500,000 line test file, anthropic.c used 155 MB while human.c used 136 MB
- This gives roughly 12% less memory usage, just from the struct optimization.

AI Debugging

I would be amiss to not mention that Claude caught a logic bug when I modified the struct initially where it was reading garbage values because I never set categoryRank but was using a second, similarly named enum called category. This has been modified and now the program runs correctly. The same happened in an accidental edition of a C++ header file Zed decided to auto include

Conclusion

Frontier models are interesting, but I still think are behind "replacing" programmers and their intuition. While they can generate extremely thorough implementations of existing code, it was very clear to me that both were too generalistic to actually implement code designed for the specific situation (at least in one prompt). I am wary of this replacing good enterprise level code, where well intentioned and tailored code makes machines written in the 90's still serve code today.

I think that understanding of the language is also generalized, meaning that AI does not intimately recall all features of the language and don't always make the best choices.

However, I considering something while reading the while(1) loop in Sonnet's code. Is good code just readable code? while(1), still being a little obtuse, removes the stdbool dependency. I think that in large code bases, using bool helps reduce the error rates between different "truthy" comparisons, but if a design guide is followed, is it true that at the end of the day, well written code is code that is readable and maintainable, instead of the best?

All of this being said, Claude Code did contribute a lot to my workflow on this assignment, to give proper diclosurement. To make sure I implemented the correct parts of the syllabus, you can find the reviews I made it do in the llm_review folder.

Thank you for reading, and I hope that my analysis was informative and insightful. As before, this is written in both MD and PDF format. Choose whichever is more readable for you.